

NSBM Green University

BSc. Honours in Computer Networks

KERNEL-LEVEL DDOS DETECTION USING eBPF XDP

By Shanirangala Ashani Gamage

Supervised By Mr. Chamara Disanayake

Contents

1. Chapter 1 - Introduction
2. Chapter 2 - Literature Review
3. Chapter 3 - Methodology
4. References

CHAPTER 1 - INTRODUCTION

1.1 Chapter Overview

This chapter sets out the research carried out on kernel-level DDoS detection using eBPF and XDP. It gives the background that motivates the study, states the problem in general and specific terms, and identifies the research question along with the motivation, aim, and objectives that guide the rest of the work. The chapter also sketches the proposed solution, lists the resources it needs, and marks out the boundary of what the project does and does not attempt.

1.2 Problem Background - What Has Actually Happened?

Denial of Service attacks are not new, but they have grown sharply in scale, frequency, and sophistication over the past two years. According to Cloudflare's threat intelligence report, in 2025 they blocked 47.1 million DDoS attacks on their network, up from 22.1 million in 2024, and the first quarter of 2025 alone accounted for 96 percent of all DDoS attacks blocked throughout the whole of 2024. A single attack, peaking at 31.4 terabits per second and 14.1 billion packets per second, lasted just 35 seconds before being automatically mitigated, as part of an internal campaign dubbed "The Night Before Christmas" that occurred in December 2025. The botnet behind it, dubbed Aisuru-Kimwolf, is believed to consist of anywhere from one to four million "smart" Internet of Things devices, most of them Android-based TVs, and it can deliver simultaneous packet-rate and bandwidth attacks across both TCP and UDP.

These numbers matter to this dissertation in two ways. First, the sheer volume of present-day attacks makes it clear that detection at the application layer, or even within the standard kernel network stack, is structurally unable to keep up - by the time an attack reaches a userspace process, or even a fully-formed kernel socket buffer, the system may already be overloaded. Second, the growing share of hyper-volumetric and protocol-level floods, together with the persistence of application-layer floods that mimic trusted users, makes a static rule unreliable: traffic from a popular promotional campaign can look almost identical to traffic from a flood attack in one month, and quite different from it the next. It is this second observation - the difficulty of telling a genuine surge apart from a malicious one, and doing so within microseconds - that shapes the specific research gap this chapter builds towards.

1.3 Problem Statement

Network operators today face a genuine dilemma. The sooner a bad packet can be dropped, the less CPU time an attacker is able to steal from the system, because the packet never reaches the layers of the operating system that would otherwise have to process it. The eBPF/XDP subsystem of the Linux kernel is a strong answer to this half of the problem: verified, sandboxed programs are attached directly at the network driver hook, so packets can be classified and dropped before the kernel even allocates a full socket buffer structure for them. At the same time, the very features that make eBPF/XDP safe and fast - a small maximum instruction count, a limited per-program stack, and no unbounded loops - make it hard to place any real intelligence inside that hook. Published systems tend to fall into two camps as a result,

and both camps share a shortcoming this dissertation tries to correct.

1.3.1 General Problem

In essence, this paradox forces organisations to choose between speed and intelligence. Static, threshold-based filters compiled into XDP - for example, dropping any source IP whose packet rate crosses a fixed number - are cheap enough to run on every packet, but cannot tell a real traffic surge (a flash sale, a viral post, a major sporting result) from an attack, since both produce the same sudden spike against the same fixed number. On the other hand, anything that needs the model itself to change, which in turn requires an update to the eBPF program, is disruptive to carry out while the system is actively defending live traffic; decision trees and quantised neural networks can be trained to recognise far more nuanced attack patterns, but only offline, before the program is loaded. Either way, the defence ends up acting too simply or too slowly for the pace of real traffic and real attackers.

1.3.2 Specific Problem

At the level of this specific research domain, the shortfall can be stated more precisely. Three trends recur across the recent eBPF/XDP DDoS literature. The mitigation results reported for static, rate-based systems tested against synthetic flood traffic such as hping3 look good against a flood, but none of the studies reviewed test the detection logic against a burst of traffic that is deliberately legitimate, so the chance that a real user gets blocked is never actually measured. Some machine-learning systems compiled around eBPF - decision-tree and quantised neural-network classifiers among them - report strong accuracy figures, mostly above 95 percent, but these figures come from labelled attack datasets such as CIC-IDS-2017 and CICDDoS2019, and the models that are actually deployed stay static once trained. The single closest piece of work, a 2025 framework that builds adaptive, dynamically-thresholded detection logic directly into OpenFlow switches, does solve the adaptivity problem, but only by recalculating a statistical threshold rather than adjusting a learned model, and it depends architecturally on an SDN controller, which limits how it could be used in a host-level, controller-independent deployment.

None of these features is individually well supported by published work, and the combination of all three appears nowhere: (a) a lightweight, two-tier detection pipeline in which only ambiguous traffic is escalated from a cheap statistical pre-filter to a small machine-learning classifier; (b) an online mechanism by which a userspace control loop can adjust the escalation

threshold that governs when traffic reaches that classifier, without recompiling or reloading the XDP program; and (c) an evaluation methodology that explicitly measures false positives against legitimate flash-crowd traffic, rather than against attack traffic alone. The gap this dissertation targets in the research literature is that combination.

1.4 Research Question

How can a kernel-level eBPF/XDP DDoS detection system dynamically adjust its detection threshold without reloading the running program, while preserving the accuracy of attack detection and significantly lowering false positives when legitimate network traffic suddenly increases?

This primary question is answered in several ways across the dissertation, and it splits into two sub-questions: what architecture is needed to let a userspace control process safely and efficiently update kernel-resident detection parameters without simply moving the problem back up into userspace, and what evaluation methodology is needed to show, with evidence, that adaptivity genuinely changes the trade-off between attack detection and false-positive rate, rather than just shifting where that trade-off is made.

1.5 Research Motivation

This research is not only practical, it is personal as well. From a practical point of view, the statistics cited in Section 1.2 make it clear that DDoS has stopped being an occasional event and has become, in the words of Zayo's 2026 Cybersecurity Insights Report, "a permanent and highly disruptive reality for all organisations." Akamai and Cloudflare are enterprise-grade scrubbing providers, but small and medium-sized organisations face the same hyper-volumetric and adaptive attacks that large organisations are working to combat, usually without the budget for that kind of protection. A lightweight, self-tuning, host-level defence that can be deployed on an ordinary Linux server, without needing a Software-Defined Networking controller or a dedicated scrubbing centre, therefore has direct practical value.

On a more personal level, the appeal of the topic lay in the chance to work at the actual interface between operating-systems research and applied security. What started as a fairly broad idea - "DDoS detection using eBPF/XDP" - and the wish to work inside the Linux kernel while engaging with a live and growing security problem, is what narrowed down into the

specific research gap identified above.

1.6 Research Aim

This work aims to design, implement, and empirically assess a two-tier, online-adaptive DDoS detection and mitigation solution built on eBPF/XDP, one that suppresses false positives on legitimate flash-crowd traffic while maintaining detection accuracy against representative DDoS attack traffic, and to show through empirical evaluation that this solution outperforms existing static-threshold baselines.

1.7 Research Objective

1.7.1 To identify

To identify the limitations of existing eBPF/XDP approaches to DDoS detection, and among these, the issues around adaptivity and the way these systems are tested against legitimate traffic surges.

1.7.2 To analyse

To compare a statistical threshold approach against a compiled machine-learning approach under the constraints imposed by the eBPF verifier, in order to design a hybrid two-tier architecture.

1.7.3 To design and develop

To build a working prototype that combines a lightweight machine-learning classifier, invoked selectively rather than on every packet, with a low-cost, kernel-resident statistical pre-filter at the XDP hook, together with a userspace control-loop daemon that can change the detection threshold via a BPF map without reloading the XDP program.

1.7.4 To evaluate

To experimentally test the prototype against both synthetic DDoS attack traffic and deliberately generated legitimate flash-crowd traffic, measuring detection accuracy, false-positive rate, throughput, and CPU overhead, and to compare the results against a static-

threshold baseline.

1.8 Rich Picture of the Proposed Solution

The proposed architecture can be thought of as two co-operating parts that sit either side of the kernel/userspace boundary. The XDP program is attached to the network interface driver and receives every packet before the kernel builds any further structures for it. For each IPv4 packet, it performs one cheap, bounded operation: it increments a per-source-IP counter held in a BPF hash map and compares that count against a threshold held in a separate BPF array map, rather than one hard-coded into the program. If the count has reached the threshold, the packet is dropped immediately with `XDP_DROP`; otherwise it is passed on with `XDP_PASS`. This part of the design is deliberately minimal - a lookup, an increment, and a comparison - so that it stays well inside the instruction and complexity limits the eBPF verifier imposes.

The second part is a userspace daemon that manages the pipeline alongside the kernel. It periodically reads the aggregate statistics out of the BPF maps, works out a new baseline for "normal" traffic using an exponentially weighted moving average, and writes an updated threshold back into the BPF map that the kernel program reads on the very next packet. Because this value can be changed from userspace at any time without unloading the program, the detection logic that applies at three in the morning, when traffic is quiet, can be different from the logic in place during a lunchtime sale, and the transition between the two happens automatically.

Sources whose counts sit in an ambiguous band - busy enough to be suspicious, but not yet over the hard threshold - are escalated to a small decision-tree classifier. Unlike the statistical pre-filter, this classifier is not compiled into the XDP program itself; it runs in userspace and is invoked only for that ambiguous minority of sources, so the more expensive classification step never sits on the packet path. This mirrors a design choice also made by comparable published systems - the SmartX Intelligent Sec framework, for example, keeps its BiLSTM-based classifier at the user level for the same reason, describing the trade-off as a limitation to be revisited in future work rather than a flaw that undermines the contribution. The classifier is trained offline, and its output feeds back into the same userspace control loop that manages the threshold, so that a consistently-suspicious source can influence how quickly the threshold reacts.

A separate evaluation harness, conceptually apart from the running defence but central to the research contribution, replays both attack traffic - generated with hping3 - and synthetic flash-crowd traffic that mimics a genuine burst of legitimate connections from many distinct sources, and logs the resulting detection accuracy and false-positive behaviour for each scenario.

1.9 Resource Requirements

1.9.1 Hardware

- A Linux test server, either a physical machine or a cloud VM with kernel-level access, running a minimum kernel version of 5.10 for stable support of the required BPF map types and XDP hooks.
- A second machine, or a separate set of cloud instances, to act as the traffic generator for attack and flash-crowd test traffic, kept physically or logically separate from the system under test so that the generator's own CPU load does not contaminate the measurements taken on the host being defended.
- Where available, a network interface card known to support native XDP mode rather than the slower generic/SKB mode, for accurate throughput figures.

1.9.2 Software

- Headers and libraries for compiling and loading eBPF/XDP programs on the Linux kernel, via the BCC toolchain.
- clang/LLVM for compiling C programs down to BPF bytecode, and bpftool for inspecting loaded programs and BPF maps.
- A userspace control-loop daemon written in Python, using BCC's Python bindings, to read and write BPF map values.
- The CIC-IDS-2017 dataset for offline model training and as a source of labelled attack and benign traffic for evaluation.
- Traffic generation and replay tools, principally hping3 for attack traffic, together with a purpose-built Python generator for legitimate flash-crowd traffic.
- Standard Python data-analysis tooling - pandas, scikit-learn, and matplotlib - for offline model training and result analysis.

1.10 Project Scope

In Scope

- Detection and mitigation of representative volumetric and protocol-level DDoS patterns, such as SYN floods and UDP floods, at the host level using eBPF/XDP.
- Design and implementation of a two-tier detection pipeline within the constraints of the eBPF verifier, combining a statistical pre-filter with a selectively-invoked machine-learning classifier.
- Experimental testing using synthetic attack traffic and synthetic legitimate flash-crowd traffic, run on a host or a small testbed.

Out of Scope

- Mitigation of encrypted, low-rate, or fully application-layer (Layer 7) attacks that closely mimic legitimate HTTP/HTTPS request semantics; this is left as future work.
- Distributed or multi-node coordination of detection state across a fleet of machines; this dissertation addresses single-host adaptivity only.
- Integration with a Software-Defined Networking controller or OpenFlow switches; the proposed system is intentionally host-level and controller-independent.
- Production-grade deployment, large-scale load testing beyond the available infrastructure, or commercial packaging of the prototype.

1.11 Chapter Summary

This chapter placed the dissertation in the context of the rapid growth in the volume and sophistication of DDoS attacks, and used that context to motivate a precise problem statement: existing eBPF/XDP detection systems are either fast but static, or accurate but frozen once deployed, and none of the reviewed literature tests how these systems behave when faced with a legitimate traffic surge. A single research question, a supporting aim, and four research-specific objectives were drawn from this problem statement, alongside a description of the proposed two-tier, online-adaptive solution, the resources it requires, and a clear boundary of what the project will and will not attempt. Chapter Two now turns to a detailed review of the literature that this problem statement is built on.

CHAPTER 2 - LITERATURE REVIEW

2.1 Chapter Overview

This chapter reviews the published literature relevant to kernel-level DDoS detection with eBPF and XDP. The chapter is structured to build an argument rather than simply list every paper in turn: it moves from the overall context of the domain, through a comparative review of existing systems that includes the researcher's own judgement on their suitability to the present problem, to a closer technological analysis of the algorithms, designs, and workflows those systems use. It ends with a reflection that draws the threads together and restates the research gap identified in Chapter One against the literature that supports it.

2.2 Conceptual Map of the Literature

To help the reader navigate the chapter, the literature has been arranged into three layers, summarised in Table 2.1. The first layer covers domain-level background: the nature of DDoS attacks and the pre-eBPF mechanisms used to mitigate them. The second layer covers current eBPF/XDP-based systems, grouped loosely by where they are deployed - edge/IoT, cloud/data-centre, container/Kubernetes, and Software-Defined Networking. The third layer sits apart from deployment context and instead looks at the algorithm, architecture, and workflow choices these systems make. More weight is given to this third layer in what follows, because it is at this level, rather than at the level of "which environment was tested," that the research gap identified in Chapter One actually sits.

Layer	Focus	Representative Sources
Domain background	DDoS taxonomy; pre-eBPF kernel filtering (netfilter/iptables, kernel-bypass approaches)	Abranches et al. (2021); StationX (2026)
Existing systems	eBPF/XDP DDoS systems by deployment context	Tolay (2025); IEEE (2025); MDPI (2023); Elzoghbi and He (2025)
Technological	Algorithms, architecture, and	Anand et al. (2025); Hara and Sasabe

Layer	Focus	Representative Sources
themes	workflow patterns across systems	(2024); Farasat et al. (2024); Zheng and Zhang (2026)

Table 2.1: Conceptual map of the literature

2.3 Domain Overview

DDoS attacks generally fall into three types. The most common volumetric attack is a SYN flood; the most common protocol attack is an amplification attack, such as a CLDAP reflection; and the most common application-layer attack is a flood of well-formed requests that is difficult to tell apart from legitimate load.

Before eBPF, packet filtering in the Linux kernel was mostly handled through netfilter and its front-ends, iptables and nftables. These are flexible tools, but they were never built for high-frequency, programmable, per-packet decision logic at line rate, and under heavy traffic the cost of walking netfilter's rule chains becomes a real bottleneck. An alternative approach, kernel-bypass frameworks such as Intel's DPDK, sidesteps the kernel entirely and hands the network interface over to a userspace application, which gives very high throughput but at the cost of the kernel's normal device drivers, security boundaries, and ease of integration with the rest of the operating system. eBPF and XDP sit in between these two extremes: packet processing stays inside the kernel, keeping its safety and integration benefits, but the program attaches early enough in the receive path to avoid much of netfilter's overhead. It is this middle ground that makes the technology attractive for DDoS defence, and it is the foundation every system discussed in the rest of this chapter builds on.

2.4 Existing Systems, Frameworks and Designs

This section summarises seven systems published between 2021 and 2026 that use eBPF/XDP specifically for denial-of-service detection and mitigation, and evaluates each against the problem this dissertation addresses.

Abranches et al. describe a network-monitoring system rather than a DDoS-specific one, but it introduces an idea reused in almost every later DDoS-specific paper: combine several monitoring tasks into a single eBPF program to avoid duplicating per-packet work, and only

run more expensive userspace analysis once a cheap in-kernel filter has flagged something worth a closer look. This "filter cheaply, escalate selectively" idea is directly relevant to the two-tier architecture proposed in Chapter One, and in this researcher's view it is an idea that most of the subsequent DDoS-specific papers have not picked up, since they mostly apply a single, generic detection mechanism uniformly to all traffic.

Tolay presents a rate-based XDP mitigation approach aimed specifically at resource-constrained IoT edge devices, tested against a 100 Mbps flood on a Raspberry Pi 4 and reporting over 97 percent mitigation effectiveness. This is well suited to its stated environment, but it does not transfer directly to the present research, because the published evaluation does not include a legitimate traffic-surge scenario, and it relies on a single, fixed-rate threshold, leaving the question of false positives completely open. A cloud-data-centre paper published at IEEE applies a similar rate-and-flow-tracking concept to east-west traffic inside cloud-native microservice deployments, again reporting a meaningful reduction in communication overhead compared with controller-polling approaches, but again without any flash-crowd evaluation. The same family of techniques appears again, applied to pod-to-pod traffic filtering, in a study published in MDPI Applied Sciences, which shows the approach works in both cloud and container settings but makes no change to the underlying detection logic to address adaptivity.

Moving to machine-learning-augmented systems, Anand et al. integrate eBPF with four classic classifiers - Decision Tree, Random Forest, Support Vector Machine, and TwinSVM - trained on the CIC-IDS-2017 dataset, with the Random Forest classifier reaching the highest accuracy at 99.44 percent. This is a strong accuracy figure and is treated here as a reasonable baseline for the offline-training side of the proposed prototype, though integrating the classifier with XDP for lower-latency processing is left as future work in the paper itself, and, like the systems above, the evaluation is carried out only against labelled attack data. Hara and Sasabe take this a step further, showing that a quantised integer-arithmetic neural network and a decision-tree classifier can both be compiled to run within the constraints the eBPF/XDP verifier imposes, using BPF tail calls to work around the instruction-count limit - a genuinely significant piece of engineering, and one that speaks directly to the design choices made in this dissertation's own prototype, discussed further in Section 1.8. The trade-off they describe, however, is not something that can be changed at runtime; it is a design-time choice between an in-kernel approach and an AF_XDP-offloaded userspace approach, not a live decision the system makes for itself.

Farasat et al. combine a heavier BiLSTM threat-detection module with an eBPF/XDP filtering layer in their SmartX Intelligent Sec framework, and report filtering over 2.2 million malicious packets in 15 seconds of live testing - the most operationally convincing real-time demonstration among the papers reviewed here. Notably, the BiLSTM model itself runs in userspace rather than inside the verifier, which the authors themselves list as a limitation of the current work and mark out as future work rather than presenting it as a settled design decision. This is judged here to be a reasonable and honestly-reported engineering compromise, even though it reintroduces some of the latency that XDP's earliest-possible-hook design is meant to remove, and it is a compromise this dissertation's own prototype adopts for the same reasons. Zheng and Zhang go further still, placing a convolutional neural network inside the kernel for edge traffic classification, emphasising the ability to classify traffic at runtime without duplicated effort, though at some cost to raw classification accuracy on constrained hardware.

Finally, of the literature surveyed, Elzoghbi and He is the closest single piece of work to the present research. They address the "frozen at deployment" weakness of the static systems above by embedding an eBPF/XDP program into the data path of an OpenFlow switch and recalculating detection thresholds dynamically to counter low-rate denial-of-service attacks in a Software-Defined Network. What separates the present dissertation from being a simple recreation of their work is two-fold: their adaptivity is statistical, meaning a recomputed threshold rather than a learned model, and their architecture depends entirely on an SDN controller and OpenFlow switches, neither of which forms part of the present research - assuming otherwise would undermine the goal of a system deployable on an ordinary Linux host.

2.5 Technological Analysis

2.5.1 Algorithmic Analysis

Two broad families of algorithm recur across the systems reviewed in Section 2.4. The first, statistical or rate-based detection, computes simple aggregate features - packets per second, bytes per second, SYN-to-ACK ratio - over a sliding or fixed time window, and compares the result against a threshold stored in a BPF map keyed by source IP or flow ID. This family is lightweight: a handful of arithmetic operations and a single map access per packet, which is why it is the default choice in constrained settings such as Tolay's IoT deployment, where the eBPF verifier's per-packet instruction budget is tight. Its weakness, raised repeatedly in Section

2.4, is that a threshold on its own has no concept of what counts as "normal" at a given time of day, or "normal" during a promotional event, so any system relying on a threshold alone is structurally unable to tell volume-driven legitimate traffic apart from a volume-driven attack.

The second family, learned classification, covers comparatively simple decision trees, random forests, support vector machines, quantised neural networks, and convolutional or recurrent architectures. This family is well known to be more accurate, but that accuracy comes at a real cost inside the kernel: a decision tree compiles fairly directly into a bounded sequence of branch comparisons, which is why it turns up in so many "fits-inside-the-verifier" papers, whereas a neural network needs the quantisation and tail-call workarounds that Hara and Sasabe had to engineer, precisely because the verifier will not accept the floating-point arithmetic a conventional inference engine would normally use. The implication for the proposed two-tier architecture is that its escalation-tier classifier should be a small decision tree rather than a neural network - not only because the verifier constraints favour it, but because that classifier is only ever invoked on the minority of traffic already flagged as ambiguous by the cheaper first tier, so the higher per-packet cost of a larger model is not needed in the first place.

2.5.2 Design Analysis

Architecturally, the systems reviewed differ along three dimensions. The first is single-tier versus multi-tier processing: most of the reviewed systems apply one mechanism, either a threshold or a classifier, uniformly to all traffic, with Abranches et al., outside the DDoS-specific context, and to a lesser extent Farasat et al., splitting a cheaper always-on stage from a more expensive stage that is invoked selectively. The second dimension is host-level versus controller-dependent deployment: Elzoghbi and He's SDN/OpenFlow architecture trades the visibility a centralised controller offers against the added complexity of deploying one, while the remaining architectures, including the one proposed here, operate independently on a single host, or, in the Kubernetes case, a single node. The third dimension, and the one most relevant to this dissertation, is static versus adaptive parameterisation: every system reviewed here hard-codes its thresholds and model weights at compile time, with the exception of Elzoghbi and He, who recalculate a purely statistical threshold; none of them combine a learned classifier with an update mechanism that runs in userspace and pushes changes back through BPF maps. This third dimension is exactly where the research gap sits, and it feeds directly into the userspace control-loop design described in Section 1.8.

2.6 Reflection

The literature reviewed here is concentrated mostly within the last two years, which reflects how actively the eBPF/XDP subsystem is still developing - a paper from several years earlier, even where the underlying idea has not aged badly, would likely be out of step with what the current verifier actually allows. Reading across this recent literature, the pattern flagged in Chapter One holds up under scrutiny: systems are either fast and static, accurate but frozen once deployed, or adaptive only in a purely statistical and controller-dependent sense. None of the sources reviewed combine a learned, selectively-invoked classifier with a live, host-level, BPF-map-driven update mechanism, and none of them measure false positives against a deliberately generated traffic surge. This chapter therefore confirms, with the support of the literature reviewed here, the research gap set out in Section 1.3.2, and the methodology proposed in Chapter Three is aimed directly at closing it.

CHAPTER 3 - METHODOLOGY

3.1 Research Paradigm

This research is conducted in a broadly post-positivist manner. The research question is treated as one with an objective, repeatable, measurable answer - in this case, the accuracy of detection, the false-positive rate, throughput, and CPU overhead, all collected under controlled experimental conditions - while still recognising, as a strict positivist stance would not, that the choice of which features to measure, which thresholds to use, and which traffic scenarios to test is itself a design decision shaped by the researcher's reading of the literature reviewed in Chapter Two. The claim this dissertation sets out to support is one that should be backed by quantitative, instrument-collected evidence rather than by interpretive methods such as interviews or case-study narrative, because what is being studied is a measurable behaviour of a system, not a subjective experience or a social meaning.

3.2 Research Approach

The research follows a deductive approach. The gap identified in the literature review in Chapter Two is well defined and testable: existing systems cannot combine selective machine-learning escalation, live threshold adaptation, and flash-crowd-aware evaluation without

measurably compromising attack-detection accuracy, and the proposed architecture should be able to reduce the false-positive rate on flash-crowd traffic to a measurably lower figure than existing static baselines while keeping detection accuracy at an acceptable level. The remainder of the methodology is built to test this expectation directly, rather than to build new theory inductively from open-ended observation. The engineering decisions described in Chapter One - such as choosing a decision tree over a neural network for the escalation tier, or keeping that classifier in userspace rather than compiling it into the XDP program - are not driven by a single overriding research philosophy so much as by what could realistically be built and verified within the project's timeline, and by the hard limits the eBPF verifier imposes.

3.3 Research Strategy

The overall strategy is experimental and centres on a working software prototype. This differs from a survey approach, which would rely on questionnaires or interviews, and from a pure case-study approach, which would examine an existing deployed system rather than build a new artefact. The plan instead is to construct the artefact described in Chapter One and then run repeated experiments on it under varying conditions, using the same test traffic, hardware, and measurement apparatus each time, changing only the architectural configuration under test - so that any difference observed between runs can be attributed to that configuration rather than to chance or to some uncontrolled difference in the inputs.

3.4 Fact Collection Mechanisms

Two types of information are gathered. The first is literature-derived fact, drawn from established databases and venues - ScienceDirect, IEEE Xplore, arXiv, Springer, and MDPI - searched for publications on DDoS detection, with claims cross-checked against their original source where possible, and each source's evaluation methodology documented in order to identify the specific "flash-crowd evaluation gap" this dissertation targets. The second is experiment-derived fact, gathered by instrumenting the prototype directly during testing: packet counters and timestamps read from BPF maps, CPU utilisation gathered from standard Linux tools such as `perf` and `/proc/stat`, and ground-truth labels - attack or legitimate - attached to each generated packet by the traffic-generation harness itself, so that detection outcomes can be scored automatically rather than judged by hand.

3.5 Research Methodology Execution Workflow

Workflow Stage	How It Was Addressed
3.5.1 Problem Identification	Addressed through background research into current industry threat statistics (Section 1.2), cross-checked against the literature's own stated limitations (Section 2.4).
3.5.2 Relevance Justification	Addressed through Cloudflare and industry threat-report statistics demonstrating the scale and urgency of the DDoS problem, and through the literature gap established in Section 2.6.
3.5.3 Comparative Analysis and Gap Justification	Addressed through the seven-system comparative review in Section 2.4 and the cross-cutting technological analysis in Section 2.5, concluding in the explicit gap statement of Section 2.6.
3.5.4 Define and Finalise Objectives	Addressed through the four research objectives in Section 1.7, each traceable to a specific element of the identified gap.
3.5.5 Design, Development, and Data Management	Addressed through the two-tier architecture and userspace control-loop design in Section 1.8, implemented using the toolchain in Section 1.9, with experimental data version-controlled alongside the source-code repository for traceability.
3.5.6 Evaluation and Communication	Addressed through the controlled experimental comparison described in Section 3.3, with results reported quantitatively against static baselines and communicated through the dissertation's results and discussion chapters.

Table 3.1: Research methodology execution workflow

3.6 Project Management Methodology

The agile, iterative nature of the implementation work makes the eBPF verifier's exact behaviour difficult to predict in advance when testing different feature sets or model sizes, so an agile framework built around a visible backlog and short iterations was judged more

suitable than a stage-gated framework such as PRINCE2. PRINCE2 tends to define requirements and deliverables in detail up front, which sits uncomfortably with the genuine technical uncertainty in this project - for instance, whether a given decision-tree depth will fit inside the verifier's instruction-count limit on the available kernel version is not something that can be known until it is tried. A visible, continually reprioritised task board handles that kind of uncertainty far better than a formal stage review does. The dissertation deadline of two months was fixed from the start, and the detailed ordering of tasks within that window was allowed to shift week to week depending on what the implementation revealed.

3.6.1 Project Timeline

Week	Task	Output
1	Set up Linux VM and BCC/eBPF tools; write a basic XDP program with a fixed packet-rate threshold	A simple XDP program that drops or passes packets
2	Add per-source-IP counting and threshold-based dropping at the XDP hook	A working detection program (xdp_ddos.c)
3	Build a Python backend to read kernel data and persist it to a database	A working data collector, with a MongoDB Atlas store and a local SQLite fallback
4	Train a small decision-tree model offline on attack data; add it as a selectively-invoked second detection layer	A two-tier detection pipeline (rule-based plus ML)
5	Build a REST API (FastAPI) so backend data can be requested; add live threshold updates	A working API
6	Connect the frontend dashboard to the real API instead of test data	A live dashboard showing real attack data
7	Test the system against attack traffic and	Test results and a comparison

Week	Task	Output
	normal traffic; record accuracy and false positives	table
8	Write up results; prepare the final report and demo	Final dissertation chapters and demo

Table 3.2: Project timeline

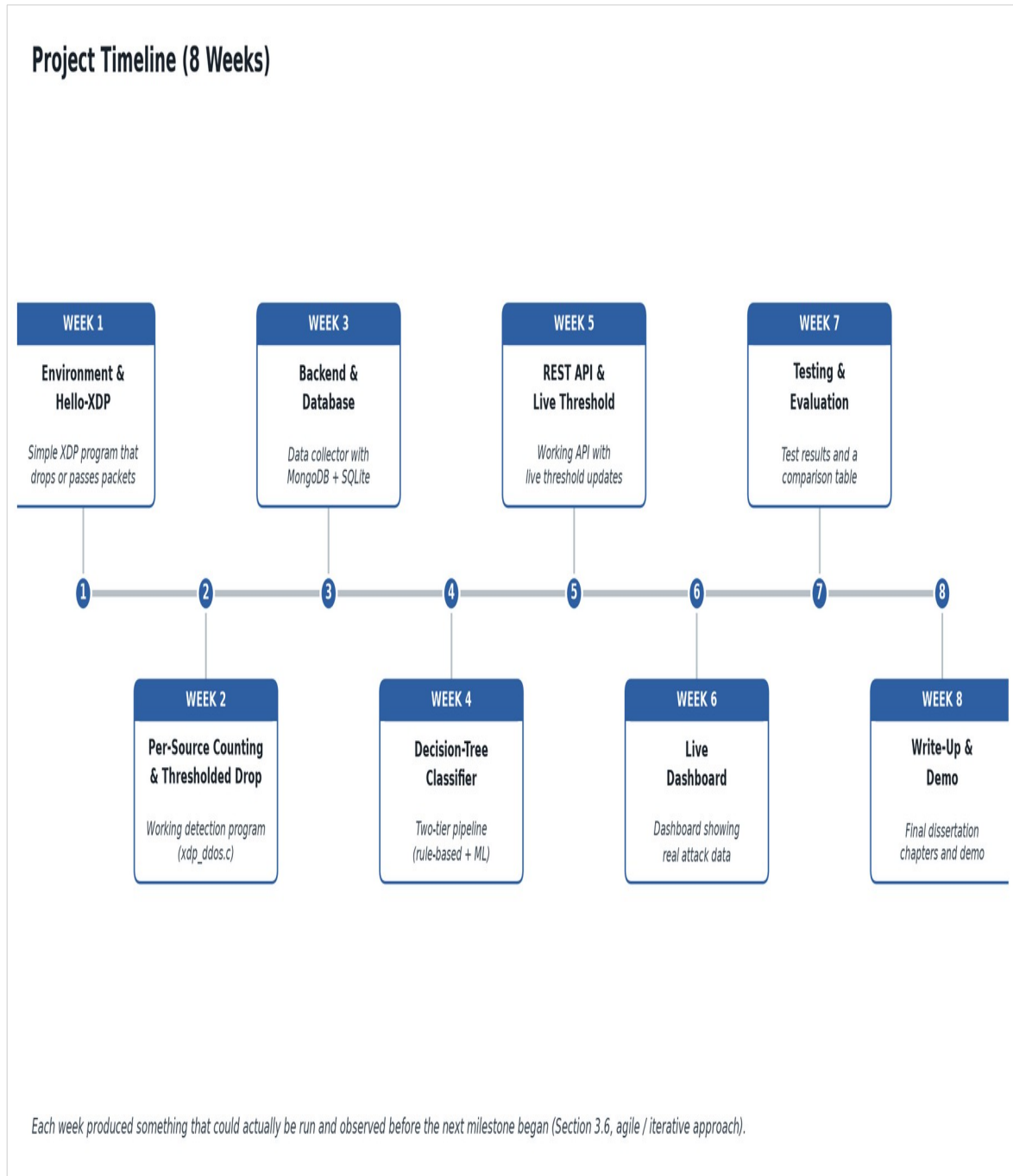


Figure 3.1: Project timeline (8 weeks).

3.6.2 Ethical Considerations

Several ethical safeguards were built into this research, given that the attack traffic involved was artificially generated and replayed in a DDoS-like manner. All attack traffic was either generated live inside an isolated test network with no access to the public internet, so that no third party could be affected, or drawn from labelled datasets intended for this purpose. No real user data was collected or used at any point in the evaluation, and all traffic treated as a

"legitimate flash crowd" was synthetically generated rather than captured from a real production system, so no personal-data or privacy concern arises. The dataset used to train the offline classifier, CIC-IDS-2017, is published, de-identified, and licensed for academic research use by the Canadian Institute for Cybersecurity, and was used strictly within the terms of that licence. Finally, because the resulting prototype is, by its nature, a description of how certain classes of static DDoS defence can be evaded - chiefly, by staying just under a fixed threshold - the contribution of the dissertation is deliberately framed around defensive adaptivity rather than published as a step-by-step attack-evasion guide, in line with responsible-disclosure practice in security research.

3.7 Chapter Summary

This chapter set out the post-positivist, deductive, and experimentally grounded methodology used to address the research question posed in Chapter One, along with the fact-collection process behind it. It showed, stage by stage, how each part of the research workflow - from problem identification through to final evaluation - is addressed by a specific part of the dissertation's design, and explained why an agile, Kanban-style approach to project management was chosen over a more formal method such as PRINCE2, given the genuine technical uncertainty of working inside the eBPF verifier's constraints. The eight-week project schedule was set out, and the ethical considerations around generating attack traffic and using a licensed dataset were discussed. The detailed design of the prototype system follows in Chapter Four.

References

1. Cloudflare, "DDoS Threat Report for 2025 Q1," Cloudflare Blog, 2025. [Online]. Available: <https://blog.cloudflare.com/ddos-threat-report-for-2025-q1/>
2. Swif, "DDoS Attack Statistics for 2026: Record Sizes, Hyper-Volumetric Attacks, and the Botnets Behind Them," 2026. [Online]. Available: <https://www.swif.ai/blog/ddos-attack-statistics>
3. M. Abranches, O. Michel, E. Keller, and S. Schmid, "Efficient Network Monitoring Applications in the Kernel with eBPF and XDP," in Proc. 2021 IEEE Conf. Network Function Virtualization and Software Defined Networks (NFV-SDN), 2021.

4. A. Tolay, "eBPF-Based Real-Time DDoS Mitigation for IoT Edge Devices," arXiv:2508.00851, 2025.
5. N. Anand et al., "Enhancing intrusion detection against denial of service and distributed denial of service attacks: Leveraging extended Berkeley packet filter and machine learning algorithms," IET Communications, vol. 19, e12879, 2025.
6. T. Hara and M. Sasabe, "Practicality of in-kernel/user-space packet processing empowered by lightweight neural network and decision tree," Computer Networks, vol. 240, 2024.
7. M. Elzoghbi and H. He, "Kernel-Level LDoS Attack Detection in SDN Networks: An eBPF/XDP Framework with Dynamic Thresholding," Computer Networks, 2025.
8. Telecom Ramblings, "DDoS Attacks Grow 70% Larger in 2025 as Enterprises Move into the Crosshairs, Zayo Reports," 2026. [Online]. Available: <https://newswire.telecomramblings.com/2026/03/>
9. StationX, "DDoS Statistics 2026: Attack Data, Records and Trends," 2026. [Online]. Available: <https://app.stationx.net/articles/ddos-statistics>
10. "Efficient DDoS Detection and Mitigation in Cloud Data Centers Using eBPF and XDP," IEEE Conference Publication, 2025.
11. "Detection of Denial of Service Attack in Cloud Based Kubernetes Using eBPF," Applied Sciences (MDPI), vol. 13, no. 8, p. 4700, 2023.
12. Y. Zheng and J. Zhang, "In-Kernel CNN Inference for Edge Devices: An eBPF-Based Approach to Low-Latency and Resource-Efficient Processing," in Algorithms and Architectures for Parallel Processing, ICA3PP 2025, LNCS vol. 16387, Springer, 2026.
13. T. Farasat, J. Kim, and J. Posegga, "SmartX Intelligent Sec: A Security Framework Based on Machine Learning and eBPF/XDP," arXiv:2410.20244, 2024.